

[Dashboard](#) / [My courses](#) / [ITB IF1210 2 2526](#) / [Praktikum 10](#) / [ADT Graph - Praktikum \(STI\)](#)

Started on	Friday, 22 May 2026, 3:01 PM
State	Finished
Completed on	Friday, 22 May 2026, 4:49 PM
Time taken	1 hour 48 mins
Grade	200.00 out of 400.00 (50%)

Question **1**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Nama File: graph_module.c

Setelah berhasil memetakan portal menggunakan *Incidence List* dengan [graph_incidencelist.c](#) dan [graph_incidencelist.h](#), anomali kini semakin tidak terkendali. Beberapa portal portal gaib yang menghubungkan *Mystery Shack* dengan area tersembunyi di ITB Jatinangor mulai tumpang tindih dan menciptakan rute ilegal. Grunkle Ford menyadari bahwa untuk menjaga stabilitas dimensi, mereka perlu melakukan pembersihan data secara presisi, yaitu menghapus jalur portal yang korup agar tidak merusak struktur kampus, serta menduplikasi peta untuk riset cadangan di dimensi lain. Ford menugaskan Dipper untuk segera mengimplementasikan fungsi `deleteEdge` agar portal yang tidak stabil dapat diputus, serta `copyGraph` untuk memastikan data riset tersimpan aman sebelum portal tersebut menghilang selamanya.

Catatan: Implementasikan fungsi `deleteEdge` dan `copyGraph` sesuai dengan spesifikasi pada header [graph_module.h](#).

C

 [graph_module.c](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	10	Accepted	0.00 sec, 1.65 MB
2	10	Accepted	0.00 sec, 1.57 MB
3	10	Accepted	0.00 sec, 1.51 MB
4	10	Accepted	0.00 sec, 1.71 MB
5	10	Accepted	0.00 sec, 1.50 MB
6	10	Accepted	0.00 sec, 1.65 MB
7	10	Accepted	0.00 sec, 1.61 MB
8	10	Accepted	0.00 sec, 1.50 MB
9	10	Accepted	0.00 sec, 1.64 MB
10	10	Accepted	0.00 sec, 1.64 MB

Question **2**

Incorrect

Mark 0.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Nama File: world_guide.c

Pecahnya anomali portal supernatural di *Mystery Shack* kini telah membentuk sebuah jaringan peta dimensi baru bernama **World Map**. Jaringan ini secara misterius menghubungkan berbagai kota gaib di Gravity Falls langsung ke titik-titik krusial di sekitar kampus **ITB Jatinangor**. Untuk memastikan keselamatan semua orang dari ancaman makhluk dimensi paralel, Dipper menulis sebuah panduan taktis di jurnalnya yang ia sebut sebagai **World Guide**.

Di dalam *World Guide* tersebut, Dipper memodelkan peta perjalanan ini menggunakan **ADT Undirected Graph dengan representasi Adjacency List**, di mana setiap lokasi bertindak sebagai simpul (Node) dan jalan penghubungnya bertindak sebagai busur berbobot jarak (SuccNode). Untuk mengoptimalkan jalur logistik dan mitigasi bencana, Dipper mendefinisikan dua entitas penting pada peta:

- **Kota Terdekat (Minimum Node):** Kota tetangga yang terhubung langsung dengan lokasi tujuan dan memiliki **jarak tempuh (weight) paling pendek/minimum**. Menemukan kota ini sangat krusial untuk mempercepat evakuasi jika terjadi gempa dimensi.
- **Kota Pusat (Central City):** Kota strategis yang bertindak sebagai hub utama karena memiliki **jumlah tetangga/koneksi jalan raya (nPred) terbanyak** di seluruh peta. Kota ini menjadi pusat pengawasan lalu lintas portal dimensi.

Sebagai asisten riset kepercayaan Grunkle Ford, kamu ditugaskan untuk melengkapi pustaka analisis navigasi ini dengan mengimplementasikan fungsi-fungsi cerdas pada modul `world_guide.c` berdasarkan spesifikasi yang telah disediakan di [world_guide.h](#). Modul dasar pembentuk peta struktural dapat diakses pada file [worldmap.h](#) dan [worldmap.c](#).

Anda hanya diminta untuk mengumpulkan file `world_guide.c` saja yang berisi fungsi manipulasi dan analisis. File pendukung seperti [boolean.h](#) sudah disediakan oleh sistem pengujian.

Batasan Masalah:

- $1 \leq \text{ID Kota} \leq 1000$
- $1 \leq \text{Total Kota pada Peta} \leq 100$
- $0 \leq \text{Jarak antar Kota (Weight)} \leq 10000 \text{ km}$
- Setiap ID Kota bersifat unik.
- Jalan penghubung bersifat dua arah (undirected) dan tidak ada jalan yang menghubungkan suatu kota ke dirinya sendiri (no self-loop).

C

 [world_guide.c](#)

Score: 0

Blackbox

Score: 0

Verdict: Compile error

Evaluator: Exact

No	Score	Verdict	Description
	0	Compile error	gcc -lm -o driver driver.c world_guide.c worldmap.c world_guide.c: In function 'deleteNode': world_guide.c:62:23: warning: implicit declaration of function 'SUCC' [-Wimplicit-function-declaration] if (SUCC(currE) == currNode) { ^~~~ world_guide.c:62:35: warning: comparison between pointer and integer if (SUCC(currE) == currNode) { ^~ world_guide.c:84:31: warning: initialization makes pointer from integer without a cast [-Wint-conversion] AdrNode targetNode = SUCC(edgeToDel); ^~~~ world_guide.c:96:31: warning: implicit declaration of function 'NEXTNODENODE' [-Wimplicit-function-declaration] NEXTNODE(prevNode) = NEXTNODENODE(currNode); ^~~~~~ world_guide.c:96:29: warning: assignment makes pointer from integer without a cast [-Wint-conversion] NEXTNODE(prevNode) = NEXTNODENODE(currNode); ^ /tmp/ccdj0eUg.o: In function `test3': driver.c:(.text+0x26d): undefined reference to `insertEdge' /tmp/ccdj0eUg.o: In function `test4': driver.c:(.text+0x340): undefined reference to `insertEdge' driver.c:(.text+0x35b): undefined reference to `insertEdge' driver.c:(.text+0x376): undefined reference to `insertEdge' driver.c:(.text+0x391): undefined reference to `insertEdge' /tmp/ccdj0eUg.o:driver.c:(.text+0x445): more undefined references to `insertEdge' follow /tmp/ccCvejBG.o: In function `deleteNode': world_guide.c:(.text+0x12f): undefined reference to `SUCC' world_guide.c:(.text+0x1d3): undefined reference to `SUCC' world_guide.c:(.text+0x246): undefined reference to `NEXTNODENODE' collect2: error: ld returned 1 exit status makefile:8: recipe for target 'c' failed make: *** [c] Error 1

Question **3**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Nama File: portal_check.c

Menjelang akhir petualangan di Nangor Falls, Dipper memasang pos pemeriksaan di setiap titik portal. Setiap pos perlu mengetahui berapa portal yang keluar, berapa portal yang masuk, apakah sebuah tujuan dapat dicapai langsung, dan apakah suatu titik merupakan titik awal bebas atau titik buntu.

Pada soal ini, graph yang digunakan adalah **Graph Berarah dengan representasi Adjacency List** dari pra-praktikum. Anda tidak diminta membuat ulang ADT graph. Gunakan fungsi dan struktur dari [graphadjlist.h](#) serta implementasi [graphadjlist.c](#) yang sudah Anda kerjakan sebelumnya.

Tugas Anda:

Implementasikan fungsi-fungsi analisis dasar pada [portal_check.h](#) di dalam file [portal_check.c](#).

```
int outDegree(Graph g, int x);
int inDegree(Graph g, int x);
boolean isAdjacent(Graph g, int x, int y);
boolean isSource(Graph g, int x);
boolean isSink(Graph g, int x);
int countEdges(Graph g);
```

Ketentuan:

- `outDegree` menghitung banyaknya successor langsung dari simpul `x`. Jika `x` tidak ada, kembalikan `0`.
- `inDegree` mengembalikan `nPred` simpul `x`. Jika `x` tidak ada, kembalikan `0`.
- `isAdjacent` bernilai `true` jika terdapat busur `x -> y`.
- `isSource` bernilai `true` jika simpul `x` ada dan tidak memiliki busur masuk.
- `isSink` bernilai `true` jika simpul `x` ada dan tidak memiliki busur keluar.
- `countEdges` menghitung seluruh busur pada graph.

Format Masukan:

Sistem akan memanggil fungsi-fungsi pada [portal_check.c](#) secara otomatis. Kumpulkan HANYA file [portal_check.c](#). Sistem akan menyediakan [boolean.h](#), [graphadjlist.h](#), [portal_check.h](#), dan driver penguji. Implementasi [graphadjlist.c](#) yang digunakan adalah implementasi ADT graph dari Pra-Praktikum.

Tips Pengerjaan Lokal:

Asisten hanya memberikan file header [portal_check.h](#) dan [graphadjlist.h](#). Jika ingin mencoba di lokal, letakkan [boolean.h](#), [graphadjlist.h](#), [graphadjlist.c](#) hasil pra-praktikum Anda, [portal_check.h](#), [portal_check.c](#), dan driver buatan Anda dalam satu folder.

Kompilasi contoh:

```
gcc -o main main.c portal_check.c graphadjlist.c
```



[portal_check.c](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	10	Accepted	0.00 sec, 1.56 MB
2	10	Accepted	0.00 sec, 1.51 MB

No	Score	Verdict	Description
3	10	Accepted	0.00 sec, 1.51 MB
4	10	Accepted	0.00 sec, 1.57 MB
5	10	Accepted	0.00 sec, 1.43 MB
6	10	Accepted	0.00 sec, 1.50 MB
7	10	Accepted	0.00 sec, 1.48 MB
8	10	Accepted	0.00 sec, 1.53 MB
9	10	Accepted	0.00 sec, 1.52 MB
10	10	Accepted	0.00 sec, 1.56 MB

Question **4**

Not answered

Marked out of 100.00

Time limit	1 s
Memory limit	64 MB

Nama File: portal_sync.c

Episode finale praktikum akhirnya tiba. Portal-portal terakhir di Nangor Falls mulai berdenyut tidak serempak: beberapa titik hanya bisa dicapai dalam satu lompatan, beberapa perlu satu titik perantara, dan beberapa pasangan portal baru stabil jika memiliki hubungan dua arah.

Dipper membutuhkan modul sinkronisasi untuk membaca pola koneksi tersebut. Graph yang digunakan tetap **Graph Berarah dengan representasi Adjacency List** dari pra-praktikum. Anda tidak diminta membuat ulang ADT graph. Gunakan [graphadjlist.h](#) serta implementasi [graphadjlist.c](#) yang sudah Anda kerjakan sebelumnya.

Tugas Anda:

Implementasikan fungsi-fungsi pada [portal_sync.h](#) di dalam file [portal_sync.c](#).

```
boolean canReachInOneStep(Graph g, int start, int target);
boolean canReachInTwoSteps(Graph g, int start, int target);
int countSharedSuccessors(Graph g, int a, int b);
int countTwoWayPairs(Graph g);
```

Ketentuan:

- `canReachInOneStep` bernilai `true` jika terdapat busur `start -> target`.
- `canReachInTwoSteps` bernilai `true` jika terdapat simpul perantara `mid` sehingga ada busur `start -> mid` dan `mid -> target`.
- `canReachInTwoSteps` boleh bernilai `true` walaupun `start -> target` juga ada, selama rute dua langkahnya ada.
- `countSharedSuccessors` menghitung banyak simpul yang menjadi successor langsung dari `a` dan juga successor langsung dari `b`.
- `countTwoWayPairs` menghitung pasangan simpul berbeda `{u, v}` yang memiliki busur dua arah `u -> v` dan `v -> u`. Setiap pasangan dihitung satu kali saja.
- Loop seperti `x -> x` tidak dihitung sebagai pasangan dua arah.

Format Masukan:

Sistem akan memanggil fungsi-fungsi pada `portal_sync.c` secara otomatis. Kumpulkan HANYA file `portal_sync.c`. Sistem akan menyediakan [boolean.h](#), [graphadjlist.h](#), [portal_sync.h](#), dan driver pengujian. Implementasi `graphadjlist.c` yang digunakan adalah implementasi ADT graph dari Pra-Praktikum.

Tips Pengerjaan Lokal:

Asisten hanya memberikan file header `portal_sync.h` dan `graphadjlist.h`. Jika ingin mencoba di lokal, letakkan `boolean.h`, `graphadjlist.h`, `graphadjlist.c` hasil pra-praktikum Anda, `portal_sync.h`, `portal_sync.c`, dan driver buatan Anda dalam satu folder.

Kompilasi contoh:

```
gcc -o main main.c portal_sync.c graphadjlist.c
```

C

◀ [ADT Graph - Pra Praktikum \(STI\)](#)

Jump to...